

5-Day Learning Plan

Python · Pydantic · FastAPI · PostgreSQL · Docker · asyncio

Day 1	Foundations — 12-Factor App · argparse · logging · python-dotenv · Enums
Day 2	Data Validation & API Design — Pydantic v2 · FastAPI · Uvicorn
Day 3	Databases — PostgreSQL · SQLAlchemy ORM · CRUD operations
Day 4	Docker — Containers · Compose · Backing Services pattern
Day 5	Concurrency & Integration — asyncio · gather · Full system

Work through each day in order. Every concept builds on the previous one.

THE 12-FACTOR APP — the philosophy behind everything

- The **12-Factor App** is a widely adopted methodology for building modern, maintainable, and scalable software applications. It defines twelve principles that guide how professional applications should be structured, configured, and deployed. Before writing any code in this course, every student must study this methodology — because every architectural decision you will make traces back to one of these twelve factors. Understanding the 'why' behind each principle is what separates a professional developer from someone who just copies code without understanding it.
- The four factors most critical to this course are: **Factor III — Config** (never hardcode secrets or environment-specific settings; store them outside your code); **Factor VII — Port Binding** (the application itself declares which port it listens on, dynamically, not by baking a number into the source); **Factor X — Dev/Prod Parity** (your local development environment should be as close as possible to the production environment, eliminating 'it works on my machine' problems); and **Factor XI — Logs** (applications should never manage log files themselves — they emit log events as a stream and let the infrastructure handle the rest). Read the official site carefully, then watch the IBM video for a concise visual explanation.

[Course notes — next session](#)

[12factor.net official](#)

[IBM explainer video](#)

[Beginner guide \(DEV.to\)](#)

CONFIGURATION MANAGEMENT — separating settings from source code

- One of the most common and dangerous mistakes beginners make is writing passwords, database URLs, API keys, and other settings directly into their source code. This is a serious security risk — anyone who reads your code, or browses your Git history, instantly has access to your credentials. **Configuration management** is the discipline of keeping all such environment-specific values completely outside your codebase, in a dedicated configuration file that is never shared or committed. Your application reads these values at startup, keeping the code itself clean, portable, and safe to share publicly.
- The **python-dotenv** library is the standard Python tool for this pattern. It reads a special hidden file at the root of your project and makes all the key-value pairs inside it available to your application as environment variables at runtime. This is the direct, practical implementation of the 12-Factor App's Factor III. The configuration file itself must never be committed to version control — it lives on each developer's machine and on each server separately, containing that environment's specific values. This is how real production systems are built.

[python-dotenv README](#)

[PyPI page](#)

COMMAND-LINE INTERFACES — building professional CLI tools

- **argparse** is Python's built-in standard library for building command-line tools. Professional software never asks users to type values interactively while the program is running — instead, all inputs are provided upfront as structured arguments when the program is launched. This makes tools automatable, scriptable, and easy to test. Understanding argparse is about learning how real developer tools, automation scripts, and production utilities are structured and invoked.
- **Logging** is the professional, structured replacement for print statements. Rather than scattering output calls throughout your code, a logging system lets you assign a severity level to every recorded event — DEBUG for granular development detail, INFO for normal operational messages, WARNING for unexpected-but-handled situations, and ERROR for genuine failures. The critical advantage is that you can control exactly how much

your application outputs simply by changing one configuration setting, with no code changes required. This is the direct implementation of Factor XI of the 12-Factor methodology, and it is what makes application behaviour observable in production without redeploying.

[argparse tutorial](#)

[argparse docs](#)

[Logging HOWTO](#)

[Logging cookbook](#)

PYTHON ENUMS — restricting values to a defined safe set

- An **Enumeration (Enum)** is a Python construct for defining a fixed, named set of valid values for a particular concept. Instead of allowing any arbitrary string to be passed as, say, a city name or a status code, you define an Enum that explicitly lists every value your application accepts. Any attempt to use a value outside this defined set is immediately caught and rejected — before any business logic runs. This pattern eliminates an entire category of bugs caused by typos, unexpected inputs, or inconsistent values flowing through the system. Enums are a cornerstone of defensive programming and are used extensively in professional Python codebases, Pydantic models, and FastAPI applications.

[enum docs](#)

End-of-day goal: By the end of Day 1, you should be able to explain the 12-Factor App methodology in your own words and describe why at least four of its principles matter in real software development. You should understand the difference between hardcoded configuration and environment-based configuration, know why logging is superior to print statements, and understand the purpose and value of input validation using restricted value sets. Build a simple command-line tool that demonstrates all these ideas working together — accepting arguments at launch, reading its configuration from an environment file, validating input against a fixed set of allowed values, and recording activity through a structured logging system.

Day 2

Data Validation & API Design

Pydantic v2 · BaseModel · FastAPI routes · HTTPException
· Uvicorn

PYDANTIC — the foundation of data validation in Python

- **Pydantic** is a Python library that enforces data types and validation rules automatically, using standard Python type annotations. Instead of writing manual checks to verify that a value is a string, or a number, or within a valid range, you define a data model class and Pydantic handles all validation for you. If incoming data does not match the expected shape, Pydantic raises a clear, descriptive error before the data ever reaches your business logic. This is fundamental to building reliable APIs — every request and every response should be validated through a Pydantic model rather than handled as a raw, unverified dictionary. Pydantic is also what FastAPI uses internally for all its validation, making it essential knowledge for this course.
- The core concept to master is the **BaseModel** — a class you define that describes the expected structure of a piece of data, including the name and type of each field. When you attempt to create an instance of that model with incorrect data, Pydantic immediately raises a `ValidationError` with a precise description of what went wrong and why. You should also understand how to use Optional fields for data that may be absent, and how to convert a validated model back into a plain dictionary when needed.

[Pydantic v2 docs](#)

[PyPI page](#)

[Pydantic tutorial \(11 min\)](#)

[FastAPI + Pydantic full tutorial](#)

FASTAPI — building modern, self-documenting APIs

- **FastAPI** is a modern Python web framework designed for building APIs quickly and correctly. It is built on top of Pydantic and uses Python type annotations to automatically validate requests, serialize responses, and generate interactive API documentation — all without any extra configuration. FastAPI is one of the most popular Python frameworks in production today, used by companies like Uber, Netflix, and Microsoft. Understanding FastAPI means understanding how modern backend services are structured: routes, HTTP methods, request bodies, response models, and error handling.
- The key concepts to cover are: how to define route handlers for different HTTP methods (GET for reading, POST for creating or processing); how to attach a Pydantic model as the expected request body so FastAPI validates it automatically; how to specify what shape the response should have using response models; and how to return structured error responses when something goes wrong — for example, returning a 404 status when a requested resource does not exist, or a 400 when the request data is invalid. Every route function should be asynchronous and every route should have full documentation describing its purpose, inputs, and outputs.

[FastAPI user guide](#)

[Request body — POST](#)

[Response models](#)

[HTTP exceptions](#)

[FastAPI beginner course](#)

UVICORN — the server that runs your FastAPI application

- **Uvicorn** is a high-performance ASGI server — it is the component responsible for actually running your FastAPI application and handling incoming network connections. Think of FastAPI as the engine and Uvicorn as the vehicle that carries it. When you want to start your API and make it accessible over the network, you use Uvicorn from the command line.
- A critical rule — directly tied to Factor VII of the 12-Factor App — is that the port your application listens on must be specified at launch time, not hardcoded inside your application's source code. The port is configuration, not behaviour. This means different environments (development, staging, production) can run the same application on different ports without any code changes. You should also understand the purpose of the auto-reload flag for development, the host binding setting for making the service reachable from outside the machine, and the log-level flag for controlling verbosity from the command line.

[Uvicorn docs](#)

[Settings reference](#)

[PyPI page](#)

End-of-day goal: By the end of Day 2, you should understand why data validation matters and be able to describe what Pydantic does and why it is preferable to manual validation. You should be able to build a working FastAPI application with multiple endpoints that accept structured request data, validate it automatically, return properly typed responses, and handle invalid inputs with meaningful error messages. You should also be able to start your application using the Uvicorn server from the command line and verify all endpoints using the interactive Swagger UI documentation that FastAPI generates automatically at the /docs path.

POSTGRESQL — understanding relational databases

- **PostgreSQL** is one of the world's most widely used open-source relational database management systems. A relational database organises data into tables — structured grids of rows and columns — where each table represents a distinct entity such as a customer, an order, or a product. Tables can be linked together through relationships, allowing complex queries that pull related data from multiple tables simultaneously. Before working with databases in Python, you need to be comfortable with the basics: understanding the table structure, being able to connect to a running database, inspect what tables exist, count how many records are in a table, and run simple retrieval queries. These fundamentals apply to almost every relational database system you will ever work with in a professional environment.

[PostgreSQL official tutorial](#)[PostgreSQL tutorial site](#)

SQLALCHEMY ORM — talking to databases using Python

- An **ORM (Object-Relational Mapper)** is a library that bridges the gap between your Python code and a relational database. Instead of writing raw SQL query strings scattered throughout your application, an ORM lets you interact with the database using Python classes and objects. Each database table is represented as a Python class, and each row in that table becomes an instance of that class. This approach makes database code cleaner, safer (less prone to injection vulnerabilities), and far more readable. **SQLAlchemy** is the dominant ORM in the Python ecosystem and integrates seamlessly with FastAPI.
- The most important concept to understand is the **session** — the temporary conversation between your application and the database. All queries, inserts, updates, and deletes happen through a session, and changes are only permanently saved when you explicitly commit the session. A critical architectural insight here connects back to Factor IV of the 12-Factor App: because you interact with the database through an abstraction layer like SQLAlchemy, you can switch from PostgreSQL to a completely different database system by changing a single configuration value — no business logic needs to be rewritten.

[SQLAlchemy ORM quickstart](#)[FastAPI + SQLAlchemy guide](#)[SQLAlchemy intro video](#)

CRUD & LAYERED ARCHITECTURE — organising your application properly

- **CRUD** stands for Create, Read, Update, and Delete — the four fundamental operations that any data-driven application performs. Every feature that involves data can ultimately be reduced to one or more of these four operations. Professional applications separate these operations into dedicated layers: a **schema layer** that defines the expected shape and types of incoming and outgoing data; a **data access layer** that contains all the database query logic; and a **routing layer** that handles HTTP requests and delegates to the appropriate operation. Each layer has a single, clear responsibility.
- This separation of concerns is what makes a codebase maintainable as it grows. When reading data, the application should support **pagination** — returning a limited number of records at a time rather than dumping thousands of rows at once. When fetching a specific record by its identifier, the application must handle the case where no record with that identifier exists, returning a descriptive error rather than crashing. When returning customer records, the response should include related data such as orders and payments — and if none exist, the response should contain an empty list rather than failing.

[Full CRUD with FastAPI](#)[Pydantic v2 docs](#)

End-of-day goal: By the end of Day 3, you should be able to explain what a relational database is and how tables and relationships work. You should understand what an ORM does and why it is preferable to writing raw SQL directly in application code. You should be able to build a complete, layered data access system with clearly separated responsibilities — validation schemas, database operations, and HTTP routing — that supports listing records with pagination, retrieving a single record with proper not-found handling, and creating, updating, and deleting records correctly.

Day 4

Docker — Containers, Compose & Backing Services

Docker concepts · docker-compose · Environment parity · Factor IV & X

WHY DOCKER — solving the environment problem

- One of the most frustrating problems in software development is code that works perfectly on one machine but fails on another. This happens because different machines have different operating systems, different versions of libraries, and different local configurations. **Docker** solves this by packaging an application and all of its dependencies together into a standardised, isolated unit called a **container**. A container runs identically on any machine that has Docker installed — your laptop, your teammate's laptop, and the production server all behave exactly the same way. This is the direct implementation of Factor X (Dev/Prod Parity) from the 12-Factor methodology.
- Docker also directly enables Factor IV (Backing Services) — the principle that your database, cache, or any other external service should be treated as an attachable, swappable resource rather than something physically installed on the same machine as your application. With Docker, you can spin up a full PostgreSQL database server with a single command, without installing anything on your local machine. The connection details live in your environment file, so the application code itself never cares where the database actually is.

[Factor IV — Backing Services](#)

[Factor X — Dev/Prod Parity](#)

DOCKER FUNDAMENTALS — images, containers, and the CLI

- Understanding Docker starts with two key concepts. A **Docker image** is a read-only blueprint that describes exactly what software and configuration a container should have — think of it as a recipe. A **container** is a running instance of that image — the actual meal made from the recipe. You can run multiple containers from the same image simultaneously, and each container is isolated from everything else on the host machine.
- The Docker command-line interface lets you manage the full container lifecycle: starting containers, stopping them, checking which ones are currently running, and even opening a terminal session inside a running container to inspect its contents directly. Pre-built images for common software like PostgreSQL, Redis, and Nginx are freely available and maintained officially — you simply reference them by name, and Docker downloads and runs them without any manual installation.

[Docker get started](#)

[Docker CLI reference](#)

[Docker crash course](#)

DOCKER COMPOSE — orchestrating multiple services together

- Real applications rarely consist of a single service — they typically involve an application server, a database, and potentially other components all running together and communicating. **Docker Compose** is the tool that

lets you define and manage this entire multi-service setup using a single configuration file. You describe each service, its image, its environment variables (read from your .env file, never hardcoded), and any special setup it requires — and Docker Compose starts everything together with a single command.

- PostgreSQL supports automatic database initialisation: if you provide SQL scripts in a special directory inside the container, PostgreSQL will execute them automatically the very first time it starts. This means your entire database schema and seed data can be set up without any manual steps — making the environment completely reproducible for any developer who joins the project. Once running, you can verify the setup by entering the container, connecting to the database, and inspecting the tables and their contents.

[Docker Compose docs](#)

[Official PostgreSQL image](#)

[Env vars in Compose](#)

End-of-day goal: By the end of Day 4, you should be able to explain what Docker is and why it solves the environment consistency problem. You should understand the difference between an image and a container, know how to manage containers from the command line, and be able to write a Docker Compose configuration that starts a PostgreSQL database automatically, initialises its schema from a SQL file, and reads all credentials from an environment file. You should also be able to connect to the running database inside the container and verify that your tables and data are correctly in place.

Day 5

Concurrency & Full System Integration

[asyncio](#) · [asyncio.gather](#) · Modular routing · Logging across all layers

CONCURRENCY — doing multiple things at the same time

- **Concurrency** is the ability of a program to make progress on multiple tasks without waiting for each one to finish before starting the next. In the context of a web API, this is essential: if your server can only handle one database query at a time, every other incoming request must wait in a queue. Under any real load, this becomes a bottleneck. Python's **asyncio** library provides a concurrency model based on cooperative multitasking — while one operation is waiting for a slow resource like a database, the program can switch to making progress on something else, returning when the original operation is ready. FastAPI is built on top of this model, which is why all route functions are defined as asynchronous.
- The **asyncio.gather** function is the key tool for running multiple independent operations simultaneously. Instead of querying eight database tables one after another — waiting for each result before starting the next — gather launches all eight queries at the same moment and waits for all of them to complete together. The total time is determined by the slowest single query, not the sum of all queries. This is Factor VIII (Concurrency) of the 12-Factor App in practice: designing the system so it can scale horizontally by handling more work in parallel rather than processing tasks sequentially.

[asyncio docs](#)

[FastAPI async guide](#)

[asyncio explained video](#)

[asyncio.gather docs](#)

MODULAR ROUTING — building clean, maintainable endpoint structures

- As an API grows, putting all routes in a single file quickly becomes unmanageable. **Modular routing** is the practice of organising API endpoints into separate, focused modules — each responsible for a distinct area of functionality. FastAPI provides a Router system that lets you define groups of related endpoints in isolation and

then register them with the main application. This improves readability, makes testing easier, and allows different developers to work on different parts of the API without conflicts.

- For the dashboard feature in this course, you will build individual endpoints for each database table that return a count of records, and a single aggregated endpoint that calls all of them concurrently and returns a unified summary response. Each individual endpoint should be independently usable — a client should be able to request just the customer count without triggering all the others. The aggregated endpoint exists as a convenience that uses concurrency to collect everything at once efficiently.

[FastAPI routers guide](#)

LOGGING ACROSS ALL LAYERS — making your system observable

- **Observability** is the ability to understand what is happening inside your application from the outside, by examining the outputs it produces. Logging is the primary tool for achieving this. In a layered architecture, logging should be present at every layer, not just at the top level. The database connection layer should record when connections are established or fail. The data access layer should record every query it executes and flag any unexpected results. The routing layer should record every incoming request, what was returned, and how long it took. This gives you a complete picture of the system's behaviour without needing to attach a debugger.
- The best practice is to configure logging in one central place and import that configuration everywhere it is needed — so log output is consistent, structured, and controlled by a single setting. The log verbosity level should be read from the environment configuration, allowing you to run with detailed debug output during development and switch to concise informational output in production — all without changing a line of application code.

[logging module docs](#)

[Logging cookbook](#)

CODE QUALITY — type hints, docstrings, and dependency management

- **Type hints** are annotations added to Python function signatures that describe what type of value each parameter expects and what type the function returns. They make code significantly easier to read, enable editor auto-complete, and allow tools to catch type-related bugs before the code even runs. In professional Python development, type hints are expected on every function. **Docstrings** are structured text descriptions written inside functions and classes that explain their purpose, what inputs they accept, what they return, and what errors they might raise. Good docstrings make a codebase self-documenting — another developer can understand what a function does without reading its implementation.
- **Dependency management** means explicitly recording every external library your project depends on, along with the exact version of each one. This ensures that anyone who runs your project gets precisely the same libraries you used — eliminating the class of bugs caused by version differences between environments. This is Factor II of the 12-Factor App: declaring and isolating dependencies explicitly rather than relying on whatever happens to be installed.

[typing module docs](#)

[PEP 257 — docstring conventions](#)

[Google Python style guide](#)

[requests library docs](#)

[pytest docs](#)

End-of-day goal: By the end of Day 5, you should be able to explain what concurrency means and why it matters for API performance. You should understand how to run multiple independent operations simultaneously rather than sequentially, and be able to articulate the performance difference between the two approaches. You should have a fully integrated system running — a containerised database, a FastAPI application connected to it through a properly layered architecture, modular and well-organised routing, concurrent data aggregation, structured logging at every layer, and clean code with proper type annotations and documentation throughout.

Quick Reference — All Resources

12-FACTOR APP

[Official site](#) · [Course notes](#) · [IBM video](#)

PYTHON STDLIB

[argparse tutorial](#) · [logging HOWTO](#) · [enum docs](#) · [asyncio docs](#) · [typing docs](#)

PYDANTIC & FASTAPI

[Pydantic v2 docs](#) · [FastAPI tutorial](#) · [FastAPI + SQL](#) · [Uvicorn docs](#)

DATABASE

[SQLAlchemy quickstart](#) · [PostgreSQL tutorial](#)

DOCKER

[Docker get started](#) · [Docker Compose docs](#) · [postgres image](#)

CODE QUALITY

[PEP 257 docstrings](#) · [Google style guide](#) · [pytest docs](#)
